

2026-06-10-SP5-SP6-parallel-dynamic-plan

Implementation Plan — SP5 (Parallel/ Subagent Enforcement) + SP6 (Dynamic- Flow Submodules)

Field	Value
Revision	1
Created	2026-06-10
Last modified	2026-06-10
Status	active
Status summary	PLANNING — read-only plan. NO code edits, NO repo creation, NO network git performed. All repo-creation tasks are OPERATOR-GATED.
Source roadmap	docs/superpowers/specs/ 2026-06-10-llms-access-master- roadmap.md §SP5 / §SP6
Source analysis	docs/superpowers/specs/ analysis/2026-06-10-F- parallel-dynamic-testing.md
Authority	Cascades from constitution/ + root CLAUDE.md/CONSTITUTION.md; anti- bluff §11.4 family governs every closure

Anti-bluff note (§11.4 / §11.4.123): This is a PLAN, not a claim of done work. No task below is “complete” until it carries captured runtime evidence per §11.4.5 / §11.4.69 / §11.4.107. Every task is authored RED-first (§11.4.43 / §11.4.115): a failing test reproduces the gap on the pre-fix tree, then implementation flips it GREEN with captured evidence, then a documented rollback path. Subagent-driven by default (§11.4.70 / §11.4.103). Repo-creation tasks are **OPERATOR-GATED** (§11.4.101 — irreversible/high-blast-radius external mutation).

Table of contents

- [1. Grounding — verified facts \(file:line\)](#)
- [2. SP5 — Parallel / subagent-driven enforcement](#)
 - [2.1 SP5 goal + gap restated](#)
 - [2.2 SP5 target architecture \(shared substrate\)](#)
 - [2.3 SP5 files to create / modify](#)
 - [2.4 SP5 ordered task list \(RED → impl → GREEN+evidence → rollback\)](#)
 - [2.5 SP5 stress / chaos coverage \(§11.4.85\)](#)

- [3. SP6 — Dynamic-flow submodules \(CREATE-LATER, operator-gated\)](#)
 - [3.1 SP6 goal + reuse-first decision](#)
 - [3.2 SP6 per-submodule spec table](#)
 - [3.3 SP6 files to create / modify](#)
 - [3.4 SP6 ordered task list](#)
 - [3.5 SP6 deep-research topics \(§11.4.99\)](#)
 - [4. Operator decisions required](#)
 - [5. Cross-cutting constraints](#)
 - [6. Executive summary \(12 lines\) + recommendation](#)
-

1. Grounding — verified facts (file:line)

Every anchor below was opened during this planning pass; ABSENCE is marked explicitly.

HelixCode parallel stack (self-contained inside helix_code/internal/): -
 helix_code/internal/agent/coordinator.go:13-23 — type Coordinator
 struct{ registry *AgentRegistry; tasks; taskQueue; results;
 workflowExecutor *WorkflowExecutor; circuitBreakers
 *CircuitBreakerManager; retryPolicy *RetryPolicy }.- helix_code/
 internal/agent/coordinator.go:26-50 —
 CoordinatorConfig{ MaxConcurrentTasks:10; ConflictResolution:
 ResolutionMethodVoting; EnableResilience; FailureThreshold:5 ... }.
 - helix_code/internal/agent/subagent/types.go:199-202 — type
 SubagentSpawner interface { Spawn(ctx, SubagentTask,
 llm.Provider) (<-chan SubagentResult, error); Kind() string }.
 Spawners present: inprocess_spawner.go, subprocess_spawner.go,
 manager.go, worktree_integration.go, helper_mode.go. Sentinels
 ErrSubagentTimeout/Canceled/MaxConcurrency/Recursion/
 UnknownIsolation (types.go:205-221); recursion cap env
 HELIXCODE_SUBAGENT_NO_RECURSE (types.go:230). - helix_code/internal/
 worker/ — worker_pool.go, ssh_pool.go, ssh_pool_consensus.go,
 consensus.go, distributed_manager.go, isolation.go, manager_db.go,
 memory_repository.go. Stress/chaos already present:
 worker_pool_stress_test.go, worker_pool_chaos_test.go,
 consensus_stress_chaos_test.go, consensus_election_test.go,
 consensus_fault_transport_test.go. - helix_code/internal/workflow/
 — executor.go, workflow.go, background.go, autonomy/ (controller.go,
 guardrails.go, escalation.go, permission.go, modes.go), planmode/
 (mode_controller.go, gating.go, planner_approval_test.go). Stress/chaos
 present: workflow_stress_test.go, workflow_chaos_test.go.

HelixAgent parallel stack (submodules/helix_agent/, module dev.helix.agent):
 - submodules/helix_agent/internal/agent/workflow.go:28-60 —
 WorkflowGraph{Nodes, Edges, EntryPoint, EndNodes},
 Node{ID, Name, Type, Handler, Condition, Config, RetryPolicy}, NodeType
 ∈ {agent, tool, condition, parallel, human, subgraph}, Edge. -
 submodules/helix_agent/internal/agents/swarm/swarm.go:1-30 —
 package swarm, imports digital.vasic.concurrency/pkg/safe, AgentColor,
 AgentRole. - submodules/helix_agent/internal/ensemble/ —
 background/, multi_instance/, synchronization/. - submodules/
 helix_agent/internal/planning/ — tree_of_thoughts.go, mcts.go,

hiplan.go.- submodules/helix_agent/internal/concurrency/ — semaphore.go, nonblocking.go, worker_pool.go, deadlock/.

Own-org reusable submodules already cloned + wired in .gitmodules: - submodules/agentica → git@github.com:vasic-digital/Agentica.git, module
digital.vasic.agentica.- submodules/concurrency → git@github.com:vasic-digital/Concurrency.git, module
digital.vasic.concurrency.- submodules/planning → git@github.com:vasic-digital/Planning.git, module
digital.vasic.planning.- submodules/llm_orchestrator → HelixDevelopment/LLMOrchestrator; submodules/debate_orchestrator → HelixDevelopment/DebateOrchestrator.

The D-7 gap (CONFIRMED this pass): - helix_code/go.mod require/replace lines for digital.vasic.

{containers,debate,helixqa,helixspecifier,lazy,docprocessor,llmorchestrator} exist (go.mod:6-10,194-212). grep -nE 'agentica|concurrency|planning' helix_code/go.mod → **EMPTY**. grep -rn 'digital.vasic.{agentica,concurrency,planning}' helix_code/internal helix_code/cmd → **EMPTY**. So HelixCode cannot import the own-org agentica/concurrency/planning substrate today; it re-implements all of it in internal/{agent,worker,workflow,task}. (Roadmap D-7, helix_code/go.mod.) - helix_code/go.mod also lacks replace dev.helix.agent (the HelixAgent cli/agentica substrate) — SP4/SP5 shared gap (roadmap D-7).

helix-deps.yaml schema (verified sample submodules/security/helix-deps.yaml):
schema_version: 1; deps: [...]; transitive_handling.
{recursive,conflict_resolution}; language_specific_subtree.

2. SP5 — Parallel / subagent-driven enforcement

2.1 SP5 goal + gap restated

Goal (roadmap §SP5): *All HelixCode work can be done by coordinated parallel agents; subagent-driven used maximally without harming the main stream.*

Gap (analysis-F §1.4): (1) duplication-not-reuse — HelixCode + HelixAgent each re-implement coordination/concurrency/planning, no shared contract; (2) no universal “every operation is a parallel-dispatchable agent unit” abstraction; (3) no swarm/ensemble parity in HelixCode; (4) no cross-process agent bus shared with HelixAgent.

SP5 strategy — EXTRACT not greenfield (§11.4.74): lift the shared coordination concern into a decoupled own-org module, reusing digital.vasic.

{concurrency,agentica,planning} underneath; wire BOTH consumers via replace; make subagent-dispatch the default substrate. SP5 does NOT create the SP6 flow engines — it creates the thin shared scheduler/dispatcher contract they (and existing code) plug into.

2.2 SP5 target architecture (shared substrate)

A new own-org module agent_substrate (HelixDevelopment/AgentSubstrate, module dev.helix.substrate) holding ONLY project-not-aware interfaces + a reference scheduler. Decision: extend digital.vasic.concurrency first (it already owns worker-

pool/semaphore/deadlock); promote agent_substrate to a NEW repo **only if** the contract does not fit inside Concurrency (operator decision — see §4). The four contracts (analysis-F §2.3):

```
// dev.helix.substrate (project-not-aware; no parent reach —
    CONST-051(B))
type Unit interface { ID() string; Run(ctx, *State, Input)
    (Output, error); DependsOn() []string }
type Scheduler interface { Submit(Unit) Handle; Await(Handle)
    (Result, error); Parallelism() int }
type Dispatcher interface { Dispatch(ctx, Task) (Result,
    error) } // sink: inprocess|subprocess|ssh|external-CLI
type Resolver interface { Resolve(ctx, []Result) (Result,
    error) } // voting/consensus
```

Adapters (config-injection only — CONST-051(B)): - HelixCode Scheduler backed by agent/subagent spawners (SubagentSpawner.Spawn, types.go:199) + worker SSH pool. - HelixCode Resolver backed by agent.ResolutionMethodVoting (coordinator.go:44) + worker/consensus.go. - HelixAgent Scheduler backed by internal/concurrency.WorkerPool + internal/background. - Cross-process Dispatcher sink (d) backed by HelixAgent clis/event_bus.go — the missing shared bus.

2.3 SP5 files to create / modify

Path	Action	Purpose
submodules/ agent_substrate/ (NEW repo, OPERATOR-GATED) OR submodules/concurrency/ pkg/substrate/ (extend- existing)	create	Unit/Scheduler/ Dispatcher/Resolver contracts + reference scheduler + helix-deps.yaml (CONST-054)
helix_code/go.mod	modify	add require dev.helix.substrate + replace ... => ../ submodules/ agent_substrate; add require dev.helix.agent + replace dev.helix.agent => ../ submodules/helix_agent (closes D-7)
helix_code/internal/ agent/ substrate_adapter.go	create	adapt SubagentSpawner + worker pool to Scheduler/ Dispatcher; adapt voting to Resolver
helix_code/internal/ agent/ substrate_adapter_test.go	create	unit RED→GREEN for the adapter contract

Path	Action	Purpose
helix_code/internal/agent/coordinator.go	modify	route taskQueue dispatch through Scheduler (default substrate) instead of inline goroutines
helix_code/internal/workflow/executor.go	modify	execute DAG steps via Scheduler.Submit/Await (every step a Unit)
helix_code/internal/agent/swarm/	create	swarm/ensemble parity port (roles, colored agents, consensus) reusing helix_agent agents/swarm via adapter — NOT a re-impl
submodules/helix_agent/internal/agent/substrate_adapter.go	create	adapt agent.Node + concurrency.WorkerPool to the shared Scheduler; keep agent as thin adapter
helix_code/tests/stresschaos/substrate_stress_test.go	create	§11.4.85 stress ($N \geq 10$ parallel Units, fan-out, deep DAG)
helix_code/tests/stresschaos/substrate_chaos_test.go	create	§11.4.85 chaos (kill subagent/worker mid-Unit, partition, resolver-tie)
helix_code/tests/stresschaos/stresschaos_meta_test.go	modify	add §1.1 paired mutation: planted deadlock/leak in scheduler → meta-test FAILs
helix_qa/banks/helixcode-parallel-substrate.yaml	create	HelixQA bank exercising the substrate end-to-end (real spawn)
challenges/banks/sp5-parallel-substrate-challenge.sh	create	Challenge: real multi-agent workflow completes with captured evidence
docs/superpowers/specs/2026-06-10-SP5-parallel-design.md	create	SP5 design spec (precedes implementation per roadmap §4)

2.4 SP5 ordered task list

Each task: **RED** (failing test on pre-fix tree) → **impl** → **GREEN+evidence** (captured runtime per §11.4.5/§11.4.69) → **rollback** (revert path). Subagent-driven (§11.4.70); long tests backgrounded (§11.4.89).

T5.0 — SP5 design spec + catalogue-check (§11.4.74). - RED: assert docs/superpowers/specs/2026-06-10-SP5-parallel-design.md absent (it is). - Impl: author spec; record Catalogue-Check: = extend vasic-digital/Concurrency@<sha> (reference impl) + reuse vasic-digital/Agentic@<sha> for graph nodes. - GREEN: spec present, metadata table (§11.4.61), ToC; .html/.pdf exported (§11.4.65). - Rollback: git rm the spec; no code touched.

T5.1 — Shared contract module bootstrap (OPERATOR-GATED on repo-vs-extend decision §4). - RED: unit test referencing `substrate.Scheduler` fails to compile (module absent). - Impl: create the 4 interfaces + a reference in-memory scheduler in `submodules/agent_substrate/` (or `submodules/concurrency/pkg/substrate/`); ship `helix-deps.yaml` declaring `concurrency/planning/agent` deps (CONST-054); no nested own-org chains (CONST-051(C)). - GREEN: `go test ./...` in the module PASS with captured output; standalone-buildable (CONST-051 decoupled). - Rollback: drop the package dir / abandon the unmerged repo (nothing wired yet).

T5.2 — Wire own-org substrate into `helix_code/go.mod` (closes D-7). - RED: a test importing `dev.helix.substrate` + `dev.helix.agent` from `helix_code/internal/agent` fails (`go.mod` lacks `require/replace` — verified empty this pass). - Impl: add `require+replace` for `dev.helix.substrate` and `dev.helix.agent`; `go mod tidy`. - GREEN: `cd helix_code && make verify-compile` PASS (pasted); `go list -m dev.helix.agent` resolves. - Rollback: revert `go.mod/go.sum`; `go mod tidy`.

T5.3 — HelixCode Scheduler/Dispatcher/Resolver adapter. - RED: `substrate_adapter_test.go` asserts `Submit`→`Await` of a Unit routes through a real `SubagentSpawner.Spawn` (`types.go:199`) — fails before adapter exists. - Impl: `substrate_adapter.go` maps `Unit`→`SubagentTask`, `Scheduler`→`spawner+worker pool`, `Resolver`→`ResolutionMethodVoting+worker/consensus.go`. - GREEN: adapter unit test PASS; an integration test spawns a real in-process subagent and returns a real result (captured stdout, not a mock). - Rollback: delete adapter + its wiring; coordinator falls back to inline path.

T5.4 — Make Scheduler the default coordinator dispatch path. - RED: a test asserts `Coordinator` dispatches via `Scheduler` (count `Submit` calls) — fails (today inline). - Impl: route `coordinator.go taskQueue` consumption through `Scheduler.Submit/Await`, preserving circuit-breaker/retry semantics (`coordinator.go:21-22`). - GREEN: existing coordinator unit + stress tests still GREEN (`make test ./internal/agent`); new dispatch-path test GREEN with captured evidence; §11.4.120 reconcile any gate that asserted the old inline path (rewrite, do NOT fake-pass). - Rollback: feature-flag the substrate path off (env), revert to inline consumption.

T5.5 — Workflow executor runs every DAG step as a Unit. - RED: test asserts `workflow_executor.go` submits each step to `Scheduler` — fails (today direct). - Impl: wrap each `WorkflowStep` (`agent/workflow.go:12-46`, `DependsOn`) as a Unit; honor `Optional`. - GREEN: `workflow_stress_test.go` + `workflow_chaos_test.go` still GREEN; new executor-via-scheduler test GREEN with captured DAG-completion evidence. - Rollback: revert executor to direct step execution.

T5.6 — Swarm/ensemble parity in HelixCode (no re-impl; adapter to `helix_agent`). - RED: test asserts a `HelixCode` swarm run produces role-tagged, consensus-resolved output — fails (absent in `HelixCode`). - Impl: `internal/agent/swarm/` thin adapter over `helix_agent agents/swarm+ensemble/multi_instance` via the shared `Dispatcher/Resolver`; investigate-before-reimplement (§11.4.124) — reuse, do not copy. - GREEN: swarm run of ≥3 roles completes; consensus resolves a deliberate disagreement (captured transcript under `docs/qa/<run-id>` / §11.4.83). - Rollback: delete `internal/agent/swarm/`; no other path depends on it.

T5.7 — HelixAgent side: keep `agent` as thin adapter to shared Scheduler. - RED: `HelixAgent` test asserts `agent.Node` execution routes through `dev.helix.substrate.Scheduler` — fails. - Impl: `submodules/helix_agent/`

internal/agent/substrate_adapter.go backs the WorkflowGraph executor with the shared scheduler + internal/concurrency.WorkerPool; keep public agent API stable (decoupled — CONST-051(B): no HelixCode reach injected). - GREEN: HelixAgent's own agent tests still GREEN (cd submodules/helix_agent && go test ./internal/agent/...); new adapter test GREEN. - Rollback: revert adapter; HelixAgent uses its native executor.

T5.8 — Cross-process agent bus (the missing symmetric bus). - RED: integration test drives a HelixCode Unit that dispatches to an external CLI agent via helix_agent clis/event_bus.go and back — fails (no shared Dispatcher sink (d)). - Impl: implement Dispatcher sink (d) over clis/event_bus.go + instance_manager.go; config-injected endpoints (no hardcode). - GREEN: round-trip prompt→external-agent→result with captured wire evidence (real exec, anti-bluff §11.4.98 fully automated). - Rollback: remove sink (d); other three sinks (in-proc/subproc/ssh) remain.

T5.9 — Full test-type coverage + Challenge + HelixQA bank (§11.4.135 guard per closure). - RED: HelixQA bank helixcode-parallel-substrate.yaml + challenges/banks/sp5-...sh absent; register RED guards (RED_MODE=1) reproducing the pre-substrate gaps. - Impl: author banks/challenges; wire into the standing regression-guard suite (§11.4.135). - GREEN: cd helix_qa && cmd/helixqa autonomous exercises the bank with captured wire evidence; Challenge PASS with pasted output; each closed item registers a RED_MODE=0 guard. - Rollback: deregister banks/guards; remove from suite manifest.

2.5 SP5 stress / chaos coverage (§11.4.85)

Extend the existing rich harness (worker_pool_stress_test.go, worker_pool_chaos_test.go, consensus_stress_chaos_test.go, workflow_stress_test.go, workflow_chaos_test.go) to the substrate: - **Stress:** $N \geq 100$ Units sustained (p50/p95/p99 latency captured); $N \geq 10$ parallel Submit (no deadlock/leak — reuse concurrency/deadlock); boundary (empty DAG / max fan-out / single-Unit). - **Chaos:** process-death (kill subagent/worker mid-Unit → consensus recovery); network-fault (partition SSH worker — reuse consensus_fault_transport_test.go); resource-exhaustion (FD/goroutine cap → refuse cleanly, never crash); state-corruption (resolver tie / mid-flight lock loss). - **§1.1 paired mutation** in stresschaos_meta_test.go: plant a deadlock or drop the leak-detector → meta-test MUST FAIL; restore → PASS. Every PASS cites a captured-evidence artefact path (boot_service/runtime-signature class per §11.4.69).

3. SP6 — Dynamic-flow submodules

3.1 SP6 goal + reuse-first decision

Goal (roadmap §SP6): dynamic flows + other execution models as NEW decoupled PUBLIC submodules under **HelixDevelopment**, wired into HelixCode + HelixAgent.

ABSENT as named packages in BOTH repos (analysis-F §2.1): standalone pipeline, standalone reusable dag, dataflow, event-driven/reactive flow, saga, standalone state-machine, behavior-tree. **EXISTING:** DAG workflow (HelixCode), graph/state-machine (helix_agent/agent), ToT/MCTS/hierarchical (helix_agent/planning).

§11.4.74 reuse-first ruling (this plan's recommendation): - planning_search → **REUSE** existing vasic-digital/Planning (digital.vasic.planning, already cloned at submodules/planning). NO new repo. Add HelixCode binding only. - flow_engine →

EXTEND vasic-digital/Agentic (WorkflowGraph/Node/Edge already cover typed nodes + conditional/parallel/human/subgraph — agentic/workflow.go:28-60 covers ≥80%). NEW repo only if dynamic re-planning/sub-flow nesting cannot land in Agentic (operator decision §4). - agent_mesh/swarm_kit → **LIFT** helix_agent agents/swarm + ensemble into the shared module (SP5 agent_substrate or its own repo). Likely folds into SP5 — see §4. - dag_orchestrator, pipeline_runtime, flow_dsl → genuinely NEW (no existing home) → NEW HelixDevelopment repos, **OPERATOR-GATED**.

3.2 SP6 per-submodule spec table

Proposed submodule	Org/repo (CREATE-LATER)	Module id	Verdict
flow_engine	HelixDevelopment/FlowEngine	dev.helix.flow	EXTEND Agentic first ; new repo only if it can't host dynamic re-plan
dag_orchestrator	HelixDevelopment/DagOrchestrator	dev.helix.dag	NEW (generalise HC workflow/executor.go logic out)
pipeline_runtime	HelixDevelopment/PipelineRuntime	dev.helix.pipeline	NEW
agent_mesh / swarm_kit	HelixDevelopment/AgentMesh	dev.helix.mesh	LIFT from helix_agent agents/swarm+ensemble — likely fold into SP5 agent_substrate
planning_search	REUSE vasic-digital/Planning (NO new repo)	digital.vasic.planning	REUSE (already at submodules/planning)
		dev.helix.flowdsl	NEW (optional)

Proposed submodule	Org/repo (CREATE-LATER)	Module id	Verdict
flow_dsl (optional)	HelixDevelopment/ FlowDSL		

Naming respects CONST-052 lowercase snake_case at path layer (submodules/
flow_engine/); repo display names CamelCase per host convention.

3.3 SP6 files to create / modify

Path	Action	Purpose
docs/superpowers/specs/ 2026-06-10-SP6-dynamic-flows- design.md	create	SP6 design spec (precedes implementation)
docs/research/ sp6_execution_models_20260610/	create	deep-research outputs for the 12 topics (§3.5), cited sources (§11.4.99)
submodules/flow_engine/ (NEW repo OR submodules/agentc extension)	OPERATOR- GATED create	flow runtime + helix- deps.yaml
submodules/dag_orchestrator/ (NEW repo)	OPERATOR- GATED create	DAG scheduler + helix- deps.yaml
submodules/pipeline_runtime/ (NEW repo)	OPERATOR- GATED create	pipeline + helix- deps.yaml
submodules/agent_mesh/ (NEW repo, or fold into SP5)	OPERATOR- GATED create	swarm/ensemble + helix- deps.yaml
submodules/flow_dsl/ (NEW repo, optional)	OPERATOR- GATED create	flow DSL + validator
.gitmodules	modify	add the approved new submodule entries (SSH only — Rule 3)
helix_code/go.mod	modify	require+replace each approved new module
helix_code/internal/workflow/ flow_adapter.go	create	register flow engines as the execution substrate
helix_code/internal/planner/ planning_search_binding.go	create	bind existing digital.vasic.planning (REUSE path) into HelixCode

Path	Action	Purpose
submodules/helix_agent/ internal/agentica/ flow_adapter.go	create	keep agentica as thin adapter over flow_engine
helix_qa/banks/sp6-flow-*.yaml (one per engine)	create	per-engine HelixQA banks
challenges/banks/sp6-flow-* challenge.sh (one per engine)	create	per-engine Challenges
each new submodule's tests/ + Challenges/	create	OWN standalone full matrix (CONST-050(B), CONST-051)

3.4 SP6 ordered task list

T6.0 — SP6 design spec + reuse-first ledger. RED: spec absent. Impl: author spec; record per-engine Catalogue-Check: (reuse/extend/new). GREEN: spec present with metadata+ToC, exported. Rollback: `git rm spec`.

T6.1 — Deep web research (§11.4.99 latest-source) on the 12 topics (§3.5). RED: docs/ research/sp6_execution_models_20260610/ absent. Impl: research each topic via WebFetch/MCP, cite source URLs+date; record “NO external solution found — original work” where applicable (§11.4.8). GREEN: research docs with `## Sources verified` footer. Rollback: `git rm research dir`. (No repo creation — safe to run anytime.)

T6.2 — Confirm reuse-vs-create per engine (OPERATOR-GATED §4). RED: n/a (decision gate). Impl: present the §3.2 table to operator via AskUserQuestion (§11.4.66); record decision. GREEN: operator answer captured. Rollback: n/a.

T6.3 ... T6.7 — Per approved engine (ONLY after T6.2 approval; repo-creation OPERATOR-GATED §11.4.101): - RED: a HelixCode integration test importing the engine module fails to compile (module absent). - Impl (repo path, OPERATOR-GATED): create the public repo under HelixDevelopment via gh/glab (SSH remotes only — Rule 3; no force §11.4.113); scaffold module + `helix-deps.yaml` (CONST-054); implement the interface from §3.2; ship the engine’s OWN full test matrix (CONST-050(B)) + Challenges, standalone-testable (CONST-051). - Impl (extend path — `flow_engine`→`Agentica`, `planning_search`→`Planning`): add the missing features TO the existing submodule (§11.4.74), bump pointer. - Impl (wire): add `.gitmodules` entry + `helix_code/go.mod` require+replace; create `flow_adapter.go` in both consumers (config-injection only — CONST-051(B)). - GREEN: engine standalone `go test ./...` PASS (captured); HelixCode + HelixAgent adapter integration+e2e PASS with captured runtime evidence; HelixQA bank + Challenge PASS; §11.4.135 regression guard registered. - Rollback: `revert .gitmodules + go.mod`; the unmerged new repo is abandoned (operator-owned external state — never force-delete).

T6.8 — flow_dsl (optional, OPERATOR-GATED). Only if operator approves at T6.2. Same RED→impl→GREEN→rollback shape; composes §11.4.106 docs-chain YAML contexts.

T6.9 — Full coverage close-out (§11.4.135 / §11.4.118). Discovery-pressure pass enumerating every flow subsystem exercised; each new engine has its own matrix + HelixQA bank + Challenge + regression guard before close-out.

3.5 SP6 deep-research topics (§11.4.99)

(From analysis-F §2.4 — DO NOT research in this planning pass; T6.1 runs them.) 1. LangGraph graph-execution semantics (conditional edges, checkpointing, human-in-loop) — latest API. 2. Temporal / Cadence durable-workflow + saga patterns in Go (durable execution, signals, queries). 3. Apache Airflow / Dagster / Prefect DAG-scheduler design (dynamic task mapping, partial-retry, backfill). 4. Behavior trees vs state machines vs DAGs for agent control flow — tradeoffs. 5. Actor model / supervision trees (Akka, Proto.Actor, Go `ergo`) for swarm supervision + fault isolation. 6. MCTS / Tree-of-Thoughts / Graph-of-Thoughts / MASTER framework latest papers (validate `planning/mcts.go`). 7. Multi-agent orchestration frameworks: AutoGen, CrewAI, OpenAI Swarm, MetaGPT — coordination + handoff. 8. Go concurrency-pattern correctness: structured concurrency, `errgroup`, semaphore-weighted, deadlock detection (validate `concurrency/deadlock`). 9. Workflow-as-data DSL design (YAML/CUE/HCL) + schema validation for `flow_dsl`. 10. Distributed consensus for agent voting (Raft/Paxos-lite) — relate to `worker/ssh_pool_consensus.go`. 11. Backpressure / flow-control for streaming pipelines (reactive streams) for `pipeline_runtime`. 12. `gh/ghlab` repo-creation + branch-protection + multi-upstream automation for the new public submodules.

4. Operator decisions required

#	Decision	Recommendation	Why operator-gated
OD-1	SP5 shared substrate: NEW repo <code>HelixDevelopment/AgentSubstrate</code> vs extend <code>vasic-digital/Concurrency (pkg/substrate/)</code>	Extend Concurrency (it already owns <code>worker-pool/semaphore/deadlock</code> ; §11.4.74 reuse-first). Only new-repo if the contract genuinely doesn't fit.	Repo creation is irreversible external mutation (§11.4.101)
OD-2	<code>flow_engine</code> : EXTEND <code>vasic-digital/Agentic</code> vs NEW <code>HelixDevelopment/FlowEngine</code>	Extend Agentic — <code>WorkflowGraph/Node/Edge</code> already cover $\geq 80\%$ (<code>agentic/workflow.go:28-60</code>). New repo only if dynamic re-planning/sub-flow nesting can't land in Agentic.	Avoids a duplicate flow engine (§11.4.74)
OD-3	<code>agent_mesh/swarm_kit</code> : fold into SP5 <code>agent_substrate</code> vs separate <code>HelixDevelopment/AgentMesh</code>	Fold into SP5 — swarm/ensemble is the Dispatcher/Resolver surface; a separate repo duplicates the contract.	Repo creation (§11.4.101)
OD-4	Which of <code>dag_orchestrator</code> / <code>pipeline_runtime</code> /	Create <code>dag_orchestrator + pipeline_runtime</code> (genuinely absent, clear	Net-new public repos (§11.4.101)

#	Decision	Recommendation	Why operator-gated
	<code>flow_dsl</code> are warranted NOW	consumers). Defer <code>flow_dsl</code> until the engines exist (it composes them).	
OD-5	<code>planning_search</code>	REUSE <code>vasic-digital/Planning</code> — NO new repo; add HelixCode binding only.	n/a (reuse) — just confirm

5. Cross-cutting constraints (apply to every SP5/SP6 task)

- **Anti-bluff §11.4 family** — captured runtime evidence on every closure; no metadata-only/config-only/grep-only PASS.
- **TDD §11.4.43 / RED-polarity §11.4.115** — every task RED-first on the pre-fix tree; RED_MODE polarity switch; standing regression guard §11.4.135.
- **§11.4.74 catalogue-first** — reuse/extend before new repo; Catalogue-Check: in every tracker row.
- **CONST-051 decoupling** — new modules are project-not-aware; config-injection only; no nested own-org chains (CONST-051(C)); deps reachable from root.
- **CONST-054** — every new submodule ships `helix-deps.yaml`; verified by the incorporate-submodule Challenge.
- **§11.4.113 no-force-push** — `fetch`→`merge-onto-latest-main`→`fast-forward` push for every repo/submodule; repo creation never force-deletes operator state.
- **§11.4.70 / §11.4.103 subagent-driven, ≥3 parallel streams; §11.4.89 background long tests; §11.4.88 detached push.**
- **§11.4.85 stress+chaos** on every fix; **§11.4.50** deterministic (N iterations, no flake).
- **§11.4.124** dead-code investigate-before-remove (no deleting “duplicate” coordinators without git-history proof + operator confirm §11.4.122).
- **§11.4.125 / §11.4.134** code-review-agent gate iterate-until-GO before pre-build+build.
- **CONST-044** keep `docs/CONTINUATION.md` in sync each state-advancing commit; CONST-062/063 doc-sync exports.

6. Executive summary

1. **SP5 gap CONFIRMED (file:line):** `helix_code/go.mod` has NO `require/replace` for `digital.vasic.{agentic, concurrency, planning}` (grep empty) and NO imports of them in `helix_code/internal/cmd` — the D-7 duplication is real; HelixCode re-implements coordination in `internal/{agent, worker, workflow}` while own-org modules sit unused.
2. **SP5 = extract a shared substrate, not greenfield.** New `Unit/Scheduler/Dispatcher/Resolver` contracts; reference scheduler; adapters back it with HelixCode `SubagentSpawner` (`agent/subagent/types.go:199`) + worker SSH pool + voting (`coordinator.go:44`), and HelixAgent `concurrency.WorkerPool` + `clis/event_bus`.
3. **SP5 wiring:** add `replace dev.helix.substrate + replace dev.helix.agent` to `helix_code/go.mod` (closes D-7), route `coordinator.go` + `workflow/executor.go` through the Scheduler, port swarm/ensemble parity as an adapter (not a re-impl per §11.4.124).

4. **SP5 stress/chaos** extends the existing rich harness (`worker_pool_*`, `consensus_stress_chaos`, `workflow_*` tests + `stresschaos_meta_test.go`) with substrate-level $N \geq 10$ parallel, kill-mid-Unit, partition, resolver-tie + a §1.1 planted-deadlock mutation.
5. **SP6 reuse-first verdict:** `planning_search` → **REUSE** `vasic-digital/Planning` (NO repo); `flow_engine` → **EXTEND** `vasic-digital/Agentic` (`WorkflowGraph` already $\geq 80\%$); `agent_mesh` → **fold into SP5**.
6. **SP6 genuinely-new repos:** `dag_orchestrator` (`dev.helix.dag`) + `pipeline_runtime` (`dev.helix.pipeline`) are absent in both repos with clear consumers — warrant creation. `flow_dsl` deferred until engines exist.
7. **Every SP6 engine** ships its OWN standalone full test matrix + Challenges + `helix-deps.yaml` (CONST-050(B)/CONST-051/CONST-054), wired into both consumers via `thin_flow_adapter.go` (config-injection only).
8. **All repo-creation tasks are OPERATOR-GATED** (§11.4.101 irreversible/high-blast-radius); the 5 operator decisions are enumerated in §4 (recommend: extend-not-create for substrate/flow_engine/mesh/planning; create only `dag_orchestrator`+`pipeline_runtime`).
9. **TDD + anti-bluff throughout:** every task is RED (on pre-fix tree) → impl → GREEN+captured-evidence → rollback; §11.4.135 regression guard per closure; §11.4.125 code-review gate before build.
10. **Deep-research (T6.1)** runs the 12 §11.4.99 topics into `docs/research/sp6_execution_models_20260610/` with cited sources BEFORE any engine is built — this is safe to run anytime (no repo creation).
11. **Plan deliverable only:** this pass wrote ONLY this doc — no code edited, no repo created, no network/mutating git run; all anchors cited from real `file:line` opened this session.
12. **Recommendation — CREATE vs REUSE:** REUSE `vasic-digital/Planning`; EXTEND `vasic-digital/Agentic` (`flow_engine`) + `vasic-digital/Concurrency` (substrate); FOLD `agent_mesh` into the substrate; CREATE only two new HelixDevelopment repos — `dag_orchestrator` + `pipeline_runtime` — and DEFER `flow_dsl`. Net new public repos warranted: **2** (plus an optional 3rd later).